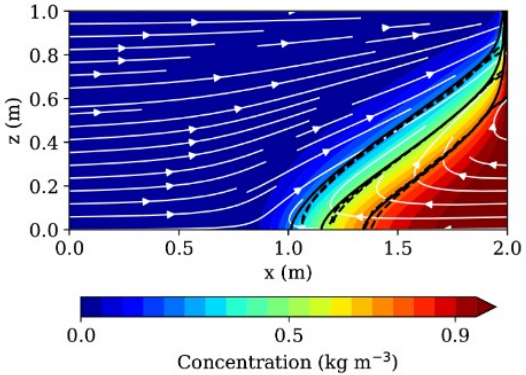
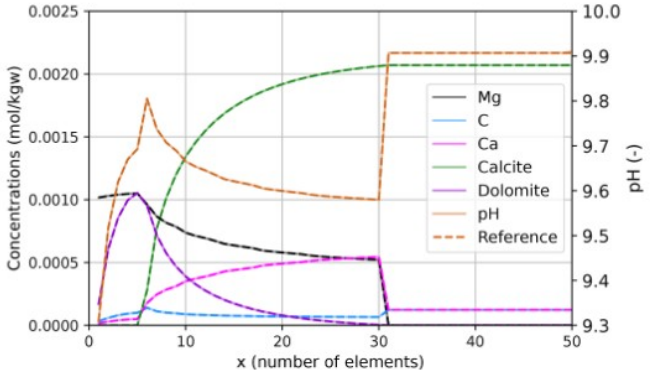
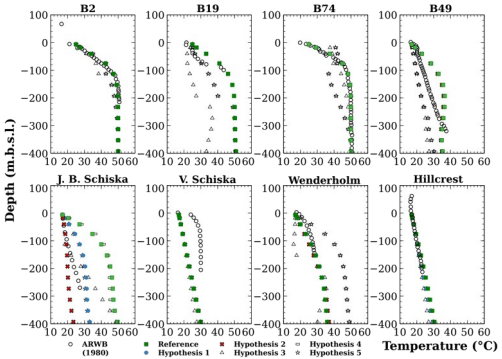
TRANSPORT Simulation Environment

Installation

Thomas Kempka
GFZ Helmholtz Centre for Geosciences / University of Potsdam
29/10/2026 - Potsdam







Software prerequisites: uv and git

uv is the Python package and project manager used by TRANSPORTSE. It replaces mamba/conda.

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh # Linux, macOS
PS> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
$ pipx install uv # alternatives: pip install uv | brew install uv
```

Documentation: https://docs.astral.sh/uv | source: https://github.com/astral-sh/uv

git (version control):

```
$ sudo apt install git # Debian/Ubuntu
$ brew install git # macOS
```

Check the installation: uv --version | git --version

Clone the TRANSPORTSE git repository

```
$ git clone https://github.com/thkempka/transportse.git
$ cd transportse
```

The repository is hosted on GitHub (public access, no account required).

pyproject.toml and uv.lock in the repository provide the complete, pinned dependency set.

Install the Python environment

uv resolves the pinned dependency set from pyproject.toml/uv.lock and installs it into a local .venv - an environment.yml is no longer required.

```
$ uv sync
```

uv init is NOT needed for TRANSPORTSE: it only scaffolds a new project (pyproject.toml with an empty dependency list) and does not add any requirements.

Reproducible install from the lock file: uv sync --frozen

uv add <package> is the command that writes new requirements into pyproject.toml and updates uv.lock.

Alternative: add individual libraries to your own project

To use the TRANSPORTSE libraries in an existing uv project, add the relevant packages to that project:

```
$ uv add joblib numba numpy phreeqpy pyevtk ipython
```

uv add writes the requirements into pyproject.toml and updates uv.lock - the equivalent of mamba install plus a manual environment.yml edit.

Development tools (e.g. pytest):

```
$ uv add --dev pytest
```

Run TRANSPORTSE

```
$ uv run python tse.py config.py
```

uv run executes the command inside the project environment - no manual activation required.

Alternatively activate the environment:

```
$ source .venv/bin/activate # Windows: .venv\Scripts\activate
$ python tse.py config.py
```